

Cours 16 — Le tri par sélection

Semaine 16 ■ Manuel Hachette ch. 6 (§ 6.3) ■ 2×1 h

Objectifs

- Dérouter et programmer le tri par sélection
- Justifier sa correction par un invariant
- Compter les comparaisons : découvrir le coût quadratique

1. L'idée : sélectionner le minimum

Pour trier un paquet de copies par note : chercher la plus petite, la mettre en premier ; chercher la plus petite du reste, la mettre en deuxième ; etc.

Sur un tableau, on trie **sur place** : à l'étape i , on cherche le minimum de la zone non triée $t[i:]$ et on l'**échange** avec $t[i]$.

$[5, 3, 8, 1] \rightarrow [1, 3, 8, 5] \rightarrow [1, 3, 8, 5] \rightarrow [1, 3, 5, 8]$ — trié.

2. Le programme

```
def tri_selection(t):  
    """Trie le tableau t en place (ordre croissant).  
  
    MODIFIE t. Ne renvoie rien.  
    """  
    n = len(t)  
    for i in range(n - 1):  
        # chercher l'indice du minimum de t[i:]  
        i_min = i  
        for j in range(i + 1, n):  
            if t[j] < t[i_min]:  
                i_min = j  
        # l'amener en position i  
        t[i], t[i_min] = t[i_min], t[i]
```

Deux boucles imbriquées : la boucle externe fixe la position à remplir, la boucle interne est la **recherche du minimum** du cours 11, restreinte à la zone non triée.

tri_selection modifie son argument et renvoie **None** — c'est écrit dans la docstring (semaine 10!). Écrire `t = tri_selection(t)` est donc un bug classique : `t` devient `None`.

3. Pourquoi c'est correct : l'invariant

Invariant : au début de l'étape i , la zone $t[0:i]$ est *triée* et contient les i *plus petits* éléments du tableau.

- Vrai au départ ($i = 0$: zone vide).
- Préservé : on ajoute en position i le minimum du reste — il est $\geq$ à tout $t[0:i]$ et $\leq$ à tout le reste.

- À la fin ($i = n - 1$) : `t[0:n-1]` trié avec les plus petits, le dernier élément est le plus grand : tout est trié.

4. Le coût : quadratique

Comptons les comparaisons `t[j] < t[i_min]` : $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$, soit environ $n^2/2$: le coût est **quadratique**, $O(n^2)$.

n	comparaisons $\approx n^2/2$	durée indicative
1 000	500 000	instantané
10 000	50 millions	quelques secondes
100 000	5 milliards	plusieurs minutes

Taille $\times 10 \rightarrow$ durée $\times 100$: c'est la signature du quadratique. Particularité de la sélection : ce coût est **le même dans tous les cas** — tableau déjà trié ou non, elle compare tout pareil.

À retenir

- Sélection : à chaque étape, minimum de la zone non triée $\rightarrow$ échange en position `i`.
- Invariant : `t[0:i]` trié et contient les i plus petits.
- Coût $n(n-1)/2$ comparaisons = $O(n^2)$, identique dans tous les cas.
- Trie **en place** : modifie `t`, renvoie `None`.